

Machine Learning Project Report:

Molecules Toxicity Classification

Batoul Ballout – Hadi Zorkot – Rasha Harb

Table of Contents

1. **Introduction**
 - 1.1 Project Overview
 - 1.2 Dataset Description
 - 1.3 Objectives
2. **Data Exploration, Preprocessing, and Feature Engineering**
 - 2.1 Initial Data Exploration
 - 2.2 Data Preprocessing
 - 2.3 Data Splitting
 - 2.4 Feature Scaling
 - 2.5 Handling Data Imbalance
 - 2.6 Feature Selection and Engineering
3. **Initial Model Training and Evaluation**
 - 3.1 Baseline Models
 - 3.2 Evaluation Metrics
 - 3.3 Results and Observations
4. **Neural Network Model**
 - 4.1 Model Overview
 - 4.2 Results and Observations
5. **KNN Classifier Model**
 - 5.1 Model Overview
 - 5.2 Results and Observations
6. **XGBoost Classifier Model**
 - 6.1 Model Overview
 - 6.2 Results and Observations
7. **Decision Tree Classifier Model**
 - 7.1 Model Overview
 - 7.2 Results and Observations
8. **Conclusion**

1. Introduction

- **1.1 Project Overview:**

The project focuses on developing a machine learning model to classify small molecules as toxic or non-toxic. This classification is essential in pharmaceutical and biomedical fields, where predicting toxicity early can significantly reduce the risk and cost of drug development. By leveraging computational methods, we aim to assist researchers in filtering out harmful compounds before conducting costly lab experiments.

- **1.2 Dataset Description:**

- The dataset used in this project is centered around the classification of small molecules as either *toxic* or *non-toxic*.
- **Number of Instances:** 159 molecules
- **Features:** 1205 numerical features per instance. The features likely represent molecular descriptors —quantitative representations of chemical properties or structural information used to characterize each molecule.
- **Label:** Binary classification – *Toxic* or *Non-Toxic*.
- **Type:** Tabular, numerical.

- **1.3 Objectives:**

To build and evaluate a machine learning model capable of accurately predicting the toxicity of small molecules. The goal is to:

- Help screen out toxic compounds early in the drug discovery pipeline
- Improve drug safety assessments
- Reduce experimental time and cost through predictive analytics

2. Data Exploration, Preprocessing, and Feature Engineering

- **2.1 Initial Data Exploration:**

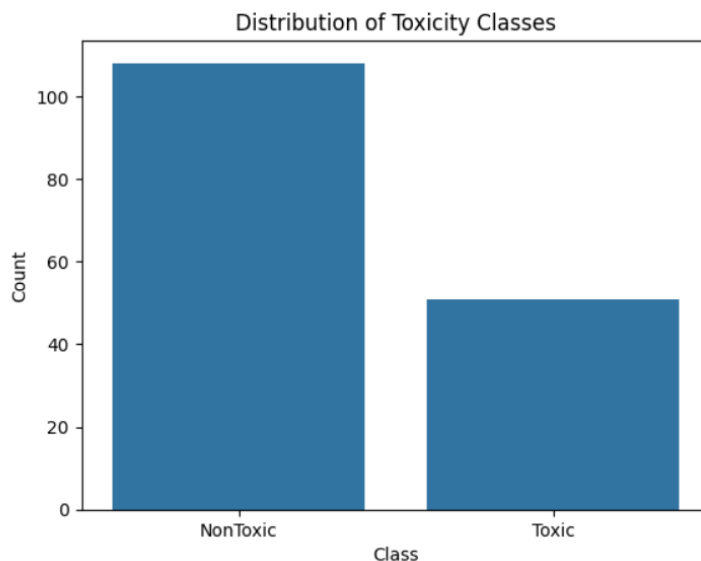
We displayed the first few rows to get an idea of the data. We also got descriptive statistics for numerical columns.

Some observations that we made:

- Most of your features are numerical (float64 and int64), which is good. However, with so many features, it will be important to consider feature selection or dimensionality reduction techniques.
- We have no missing values in this dataset.
- The descriptive statistics (df.describe()) show that the numerical features have very different ranges. This suggests that **feature scaling** will likely be beneficial for many machine learning models.

We then dropped 'Unnamed: 0' column which is redundant and checked the distribution of the target variable 'Class' and observed the following:

- The dataset is **imbalanced** where 108 instances are classified as “non-toxic” while 51 instances are considered “toxic”.



• 2.2 Data Preprocessing

- We separated our data into features (X) and target (y) which is 'Class'.
- Then, we did Label Encoding to convert the categorical target variable 'Class' into a numerical representation. (**Non-Toxic is encoded as 0, Toxic is encoded as 1**).

• 2.3 Data Splitting:

For splitting the data, we chose a small test size that ranged between 0.07 and 0.09. This decision was taken after we found that for other values, the testing set was overly optimistic, in which cross-validation was low while testing was high.

• 2.4 Feature Scaling:

We scaled the features using z-scale normalization in order to ensure similar ranges for features and quick convergence of gradient descent.

We tried 2 approaches:

```
scaler = StandardScaler()  
X_train = scaler.fit_transform(X_train)  
X_test = scaler.fit_transform(X_test)
```

○

In this approach, we first fit the StandardScaler on our training data which then **transforms** the X_train data using calculated statistics, resulting in a scaled version of our training data. Then, we fit a **new** StandardScaler on **testing data** and then **transformed** it.

```
scaler = StandardScaler()  
X_train_scaled = scaler.fit_transform(X_train)  
X_test_scaled = scaler.transform(X_test)
```

○

We used standardization and only fit the scaler on the training data. This is crucial to prevent data leakage from the test set into the training process. The scaler learns the mean and standard deviation from the training data only. We then use the same fitted scaler to transform both the training and testing data. This ensures that both datasets are on the same scale, relative to the training data's distribution.

• 2.5 Handling Data Imbalance:

We considered two oversampling techniques:

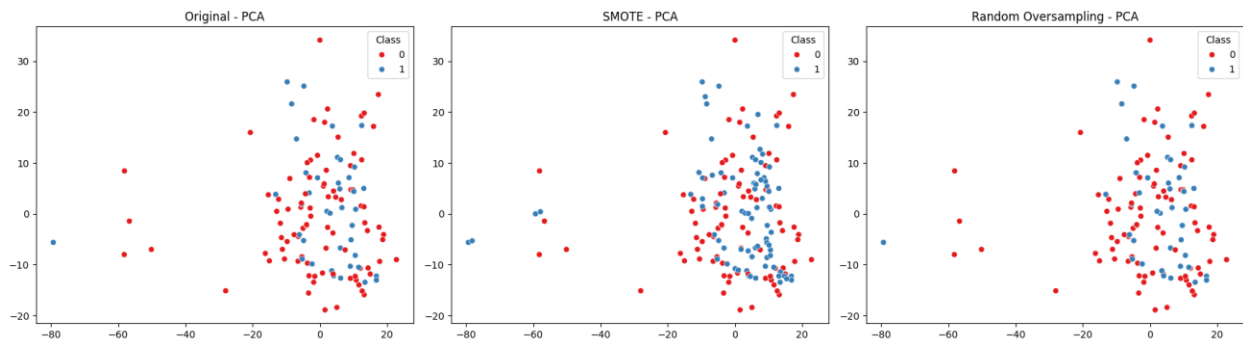
- SMOTE (Synthetic Minority Oversampling Technique)
- Random Oversampling

Why SMOTE Might Not Work Well

- SMOTE generates synthetic samples using the k-nearest neighbors of minority class instances.
- In our dataset, the classes were already close in feature space, which causes SMOTE to:
 - Create overlapping class regions
 - Potentially weaken class separability
 - Lead to confused decision boundaries in classification

To gain an initial sense of structure, we applied PCA to reduce data to 2D.

- **PCA** was applied on:
 - Original training set
 - SMOTE-augmented training set
 - Random Oversampling-augmented training set



Random Oversampling as an Alternative

- Random Oversampling was applied for comparison.
 - It duplicates existing minority class samples.
 - It does not introduce synthetic noise.
 - Preserves the original distribution better, without creating risky overlaps.

High-dimensional data cannot be visualized easily in 2D.

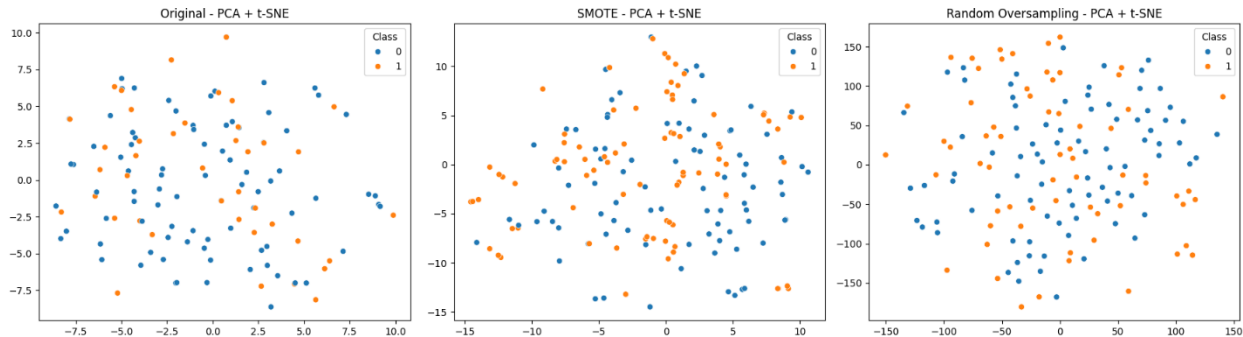
To address this, we used a two-step dimensionality reduction approach:

- Step 1: PCA to reduce dimensions to 50 (helps denoise and speed up t-SNE)
- Step 2: t-SNE to reduce from 50D → 2D for effective visualization
- This combination balances:
 - Efficiency (PCA denoising)
 - Accuracy (t-SNE preserving neighborhood structures)

PCA + t-SNE

- PCA and t-SNE were applied independently to each version of the data:
 - Original training data
 - SMOTE-augmented training data
 - Random Oversampling-augmented training data
- This avoids using a PCA basis that doesn't reflect the structure of resampled data.

- Results:
 - Clearer clusters
 - Faithful representation of changes introduced by each resampling method



• 2.6 Feature Selection and Engineering:

- Due to the extreme number of features, we decided to perform feature selection. We used “SelectKBest” since it keeps the features intact, and actually measures their correlation with the target. This is in contrast to PCA, which is more of a dimensionality reduction algorithm, which doesn’t maintain the features, and only measures variance independent of the target.
- We also tried to improve the model’s performance by performing feature selection using XGBoost and keeping only the top 15 features based on importance.

3. Initial Model Training and Evaluation

• 3.1 Baseline Models:

Before trying any model, and to have an idea of what is considered a good accuracy, we had to establish a baseline. For this reason, we used this study: <https://www.semanticscholar.org/paper/Structure-based-design-and-classifications-of-small-Gul-Rahim/5944836c47bc7d1a2b0464a9a1db94d4bc7f28ce>

The models we tried then are as follows:

- The first model we trained was a simple Logistic Regression. We Used L1 regularization to perform automatic feature selection in order to reduce the number of features further. We made sure to vary the regularization parameter in order to find the best one in GridSearch. This established a baseline for comparing to the study, as well as to compare to other models.

- The next model we trained was a Random Forest Classifier. With this model, we hoped to capture non-linear relationships in order to get better results. Using Ensemble methods also helps prevent overfitting.
- We decided to try out a Decision Tree model since that was the most successful model in the study.
- We decided to use Boosting in order to try capturing more of the relationships in the data. We made sure to keep the depth and the number of features relatively low in order to avoid overfitting as much as possible.

Note that no hyperparameter tuning was performed at this stage.

3.2 Evaluation Metrics:

We used accuracy, precision, recall, and f1-score to compare the models.

3.3 Results and Observations:

We didn't get "true" results although in some cases accuracy was considered high. This is because our test set is very small, the data seemed to be complex for it to be captured in a basic linear relationship, and the models were underfitting in some cases while overfitting in others.

4. Neural Network Model:

We conducted 3 experiments using neural network models to classify the dataset.

The experiments highlighted the challenges of working with a small and imbalanced dataset when training neural networks.

- The baseline model (Trial 1) was heavily biased towards the majority class.
- Applying class weights and adjusting the model architecture (Trial 2) showed some promise in improving the detection of the minority class, especially when the classification threshold was lowered. This trade-off between precision and recall for different classes needs to be considered based on the specific application requirements.
- Further hyperparameter tuning and architectural changes in Trial 3 did not consistently improve performance and, in some cases, led to worse results. This could be due to overfitting on the small dataset or suboptimal hyperparameter choices.

We can notice that the importance of the classification threshold is evident, as adjusting it significantly impacted the precision and recall for both classes.

Given the limitations of the small and imbalanced dataset, neural networks is not the best choice.

5. KNN Classifier Model:

We tried KNN model and got acceptable results.

For k-NN Performance (k=11):

- It gave a good cross-validation accuracy with respect to neural networks and other baseline models. It also gave a test set accuracy of 72% which suggested that KNN is generalizing much better to the unseen test data.
- We decided to fine-tune it further. We expanded the range of k values we search and also considered the weights parameter.
- While the cross-validation process favored a smaller number of neighbors (k=4), this configuration was not generalizing as well to the test set, particularly for the 'Toxic' class.
- We tried different values for p (including Manhattan and a Minkowski distance with p=3), but the best parameters found by cross-validation were still {'n_neighbors': 4, 'p': 2 (Euclidean), 'weights': 'uniform'}, which is the same as one of our previous best configurations. The test set accuracy (0.6875) and the classification report were also very similar to what we saw with that configuration. This suggested that simply changing the distance metric within k-NN is not the key to significant further improvement in our case.

6. XGBoost Classifier Model:

From our initial notebook, XGBoost was the model that was the most promising. Therefore, we decided to delve deep into it in this part and try to maximize the results we get from it.

- **Feature Scaling:**

The first step we did was feature scaling. We note that the following way makes the assumption that we have the testing set. We realize that this assumption is not a practical one to make, and we will see how this will cause some problems later.

```
scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.fit_transform(X_test)
```

- **Pipelining:**

- A major problem we faced while developing this classifier was our unfamiliarity with how to separate data preprocessing steps

- We were initially performing random oversampling on the training set, then performing k-fold cross validation on that set too.
 - This caused our CV results to be artificially inflated because the model was being evaluated on data generated from Random Oversampling, meaning it had already seen the data before.
 - We also faced similar issues when it came to scaling and validation.
 - The solution was to understand how Pipelines work in the sense that they automatically perform this separation for us.
 - When we implemented this fix, our results became a lot more consistent, albeit worse.
- **Randomized Search for Hyperparameter Tuning:**
 - We knew that we were at a large risk of overfitting, therefore, we made sure to have L1 and L2 regularization with relatively high parameters to counter this.
 - While developing this model, we also faced another issue that would prove almost impossible to solve. The search space to find the best hyperparameters was simply too large for us to feasibly traverse using our basic laptops/devices.
 - The search space for our parameters was around 32,000. We were running 150 iterations which are less than 1% of the available space. These 150 iterations took 35-40 minutes to run. It became pretty clear that moving forward, we would have to make use of as much available information as we could to give ourselves a strong starting point.
- **Results:**
 - After initially thinking that XGBoost was promising, our mistakes and limitations made it very difficult to achieve any meaningful results:
 - **CV Mean accuracy:** 62%
 - **Testing accuracy:** 47%
 - **Training accuracy:** 100%
 - It is very clear that the model is terribly overfitting, even with the high regularization parameters. This is now expected as we realize that searching the hyperparameter space is practically impossible to search.
 - We needed a strong place to start from, which is why we turned to the study we found.

7. Decision Tree Classifier:

- After discovering our mistakes with XGBoost, we realized that it would be infeasible for us to figure out the best hyperparameters using our machines.
 - Therefore, we decided to make use of the work done by the study we found, and adapt their decision tree classifier to our new, smaller dataset
- **Feature Selection:**
 - The main aim of the study was to identify the most relevant features for toxicity classification, and they had narrowed the features down to 13.

- We decided to use these 13 directly since it would have also been infeasible for us to figure them out with traditional methods like grid search or randomized search, as we saw in the XGB notebook
- We also decided not to use any oversampling methods since as we saw previously, SMOTE is unreliable, and Random Oversampling increases our chances of overfitting, which is already a high risk.

- **Class Weight Tuning:**

In the study, the formulas they used for the weights were $w_T = n/(2 \cdot n_T)$, $w_{NT} = n/(2 \cdot n_{NT})$, which gave class weights of 1:1.53, 0:0.74. We realized that our dataset is smaller by 12 examples, therefore, it would have less chances to learn from examples of the positive class. In order to combat this, we decided to perform grid search over a few possible class weight distributions, and perform CV in order to select the best one.

- **Results:**

- We got very good results using this method
 - **CV Mean Accuracy:** 77%
 - **CV Max Accuracy:** 88%
 - **Testing Accuracy:** 93%
 - **Training accuracy:** 97%
- The mean cv accuracy was very close to what the study got (78%). We believe that the differences come from the extra 12 examples that their model had the chance to train on.
- Our testing accuracy was also very high. Here, we would like to note that the validity of metrics on such small testing sets is debatable simply due to the extreme fluctuations that are caused by the size of the set. However, we are given confidence by the high CV mean score.

8. Conclusion

In this project, we developed a machine learning model to classify small molecules as toxic or non-toxic to assist in drug discovery. The dataset contained 159 instances with 1205 features. We used various data preprocessing and feature engineering techniques, including feature scaling and handling class imbalance with methods like SMOTE and Random Oversampling. Multiple machine learning models, including Logistic Regression, Random Forest, KNN, XGBoost, and Decision Trees, were tested. We encountered issues with cross-validation (CV) as it involved oversampled training data, which led to artificially inflated results and overfitting. Despite these challenges, the Decision Tree model, with feature selection and class weight tuning, achieved the best testing accuracy of 93%. The findings highlight the importance of properly handling data

preprocessing techniques and show the potential of machine learning in toxicity classification for drug safety.